


# Lecture 25

## Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

October 2, 2026

# Keeping track of local information

- When a nested procedure is seen, processing of declaration in enclosing procedure is temporarily suspended

# Keeping track of local information

- When a nested procedure is seen, processing of declaration in enclosing procedure is temporarily suspended
- A new symbol table is created when procedure declaration $D \rightarrow proc\ id; D_1; S$ is seen

# Keeping track of local information

- When a nested procedure is seen, processing of declaration in enclosing procedure is temporarily suspended
- A new symbol table is created when procedure declaration $D \rightarrow \mathit{proc}\ \mathit{id}; D_1; S$ is seen
- Entries for $D_1$ are created in the new symbol table

# Keeping track of local information

- When a nested procedure is seen, processing of declaration in enclosing procedure is temporarily suspended
- A new symbol table is created when procedure declaration $D \rightarrow \mathit{proc}\ \mathit{id}; D_1; S$ is seen
- Entries for $D_1$ are created in the new symbol table
- The name represented by `id` is local to the enclosing procedure

# Creating Symbol Table

- **mktable(previous):** create a new symbol table and return a pointer to the new table. The argument previous points to the enclosing procedure.

# Creating Symbol Table

- **mktable(previous):** create a new symbol table and return a pointer to the new table. The argument previous points to the enclosing procedure.
- **enter (table, name, type, offset):** Create a new entry

# Creating Symbol Table

- **mktable(previous):** create a new symbol table and return a pointer to the new table. The argument previous points to the enclosing procedure.
- **enter (table, name, type, offset):** Create a new entry
- **addwidth (table, width):** records cumulative width of all the entries in a table

# Creating Symbol Table

- **mktable(previous):** create a new symbol table and return a pointer to the new table. The argument previous points to the enclosing procedure.
- **enter (table, name, type, offset):** Create a new entry
- **addwidth (table, width):** records cumulative width of all the entries in a table
- **enterproc (table, name, newtable):** creates a new entry for procedure name. newtable points to the symbol table of the new procedure

# Creating Symbol Table

- **mktable(previous):** create a new symbol table and return a pointer to the new table. The argument previous points to the enclosing procedure.
- **enter (table, name, type, offset):** Create a new entry
- **addwidth (table, width):** records cumulative width of all the entries in a table
- **enterproc (table, name, newtable):** creates a new entry for procedure name. newtable points to the symbol table of the new procedure
- $P \rightarrow D$
- $P \rightarrow \quad \{t = mktable(nil);$
  $push(t, tblptr);$
  $push(0, offset)\} \quad D$

# Creating Symbol Table

- **mktable(previous):** create a new symbol table and return a pointer to the new table. The argument previous points to the enclosing procedure.
- **enter (table, name, type, offset):** Create a new entry
- **addwidth (table, width):** records cumulative width of all the entries in a table
- **enterproc (table, name, newtable):** creates a new entry for procedure name. newtable points to the symbol table of the new procedure
- $P \rightarrow D$
- $P \rightarrow \quad \{t = mktable(nil);$
  $push(t, tblptr);$
  $push(0, offset)\} \quad D$
  $\{addwidth(top(tblptr), top(offset));$
  $pop(tblptr);$
  $pop(offset)\}$

# Creating Symbol Table

- $D \rightarrow proc\ id\ D_1; S;$

# Creating Symbol Table

- $D \rightarrow proc\ id\ D_1; S;$
- $D \rightarrow proc\ id; \{t = mktable(top(tblptr));$
  $push(t, tblptr);$
  $push(0, offset)\}\quad D_1; S$

# Creating Symbol Table

- $D \rightarrow proc\ id\ D_1; S;$
- $D \rightarrow proc\ id; \{t = mktable(top(tblptr));$
  $push(t, tblptr);$
  $push(0, offset)\}\quad D_1; S\{t = top(tblptr);$
  $addwidth(t, top(offset)); pop(tblptr); pop(offset);$
  $enterproc(top(tblptr), id.name, t)\}$
- $D \rightarrow id : T$

# Creating Symbol Table

- $D \rightarrow proc\ id\ D_1; S;$
- $D \rightarrow proc\ id; \{t = mktable(top(tblptr));$
  $push(t, tblptr);$
  $push(0, offset)\}\quad D_1; S\{t = top(tblptr);$
  $addwidth(t, top(offset)); pop(tblptr); pop(offset);$
  $enterproc(top(tblptr), id.name, t)\}$
- $D \rightarrow id : T\quad \{enter(top(tblptr), id.name, T.type, top(offset));$
  $top(offset) = top(offset) + T.width\}$

# Syntax directed translation of expression into 3-address code

- $S \rightarrow id = E$

# Syntax directed translation of expression into 3-address code

- $S \rightarrow id = E \quad S.code = E.code || gen(id.place = E.place)$

# Syntax directed translation of expression into 3-address code

- $S \rightarrow id = E \quad S.code = E.code || gen(id.place = E.place)$
- $E \rightarrow E_1 + E_2$

# Syntax directed translation of expression into 3-address code

- $S \rightarrow id = E \quad S.code = E.code || gen(id.place = E.place)$
- $E \rightarrow E_1 + E_2 \quad E.place = newtmp() \; E.code = E_1.code || E_2.code || gen(E.place = E_1.place + E_2.place)$
- $E \rightarrow E_1 * E_2$

# Syntax directed translation of expression into 3-address code

- $S \rightarrow id = E \quad S.code = E.code || gen(id.place = E.place)$
- $E \rightarrow E_1 + E_2 \quad E.place = newtmp() \; E.code = E_1.code || E_2.code || gen(E.place = E_1.place + E_2.place)$
- $E \rightarrow E_1 * E_2 \quad E.place = newtmp() \; E.code = E_1.code || E_2.code || gen(E.place = E_1.place * E_2.place)$
- $E \rightarrow -E_1$

# Syntax directed translation of expression into 3-address code

- $S \rightarrow id = E \quad S.code = E.code || gen(id.place = E.place)$
- $E \rightarrow E_1 + E_2 \quad E.place = newtmp() \; E.code = E_1.code || E_2.code || gen(E.place = E_1.place + E_2.place)$
- $E \rightarrow E_1 * E_2 \quad E.place = newtmp() \; E.code = E_1.code || E_2.code || gen(E.place = E_1.place * E_2.place)$
- $E \rightarrow -E_1 \quad E.place = newtmp() \; E.code = E_1.code || gen(E.place = -E_1.place)$

# Syntax directed translation of expression into 3-address code

- $E \rightarrow (E_1)$ $E.place = E_1.place$
  $E.code = E_1.code$

# Syntax directed translation of expression into 3-address code

- $E \rightarrow (E_1)$ $E.place = E_1.place$
  $E.code = E_1.code$
- $E \rightarrow id$

# Syntax directed translation of expression into 3-address code

- $E \rightarrow (E_1)$ $E.place = E_1.place$
  $E.code = E_1.code$
- $E \rightarrow id$ $E.place = id.place$
  $E.code =$ " "

**Generate 3AC for** $a = b * -c + b * -c$